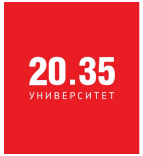
3.5. Основы SQL и СУБД. SELECT, INSERT, WHERE

3. Сети, базы данных, безопасность кода



В этом уроке

- Разберемся как хранятся большие объёмы данных
- Узнаем что такое базы данных
- Изучим где специалист ИБ может с ними столкнуться
- Узнаем какие проблемы в безопасности баз данных могут встретиться и как с ними бороться

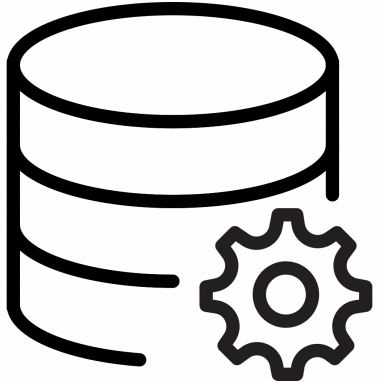
Зачем нужны базы данных?

Хранят большие объёмы информации

Структурируют данные для удобного поиска

Используются в:

- Сайтах, чатах, онлайн-магазинах
- Сканерах уязвимостей, SIEM-системах
- Учёте инцидентов ИБ, логировании



Где с БД работает специалист по кибербезопасности?

👁️‍🗨️ Анализ логов атак (хранятся в БД)

🔎 Парсинг отчетов сканеров (результаты = таблицы)

🔐 Проверка уязвимостей через SQL-запросы

🕵️‍♀️ Поиск инцидентов, корреляция событий

⚠️ Поиск следов SQL-инъекций

Что такое СУБД?

СУБД — система управления базами данных

📚 Хранит таблицы

🔄 Позволяет добавлять, удалять, искать

Примеры:

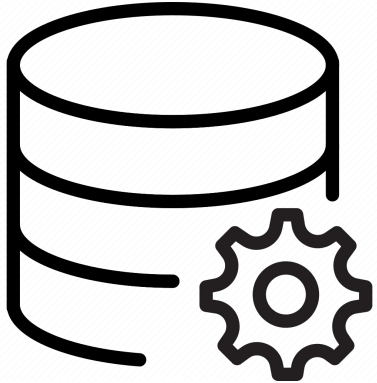
- SQLite, PostgreSQL, MySQL
- 💡 Мы будем использовать **SQLite** — встроена в Python, ничего ставить не нужно

Что такое SQL?

SQL — язык запросов к базе данных

Позволяет:

📄 Читать данные: SELECT
➕ Добавлять данные: INSERT
🗑️ Удалять/менять: DELETE, UPDATE
🔎 Фильтровать и искать

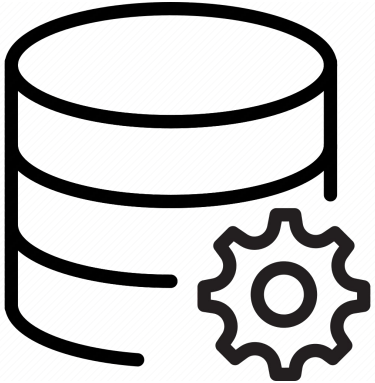


Пример таблицы

Пример таблицы “Пользователи”

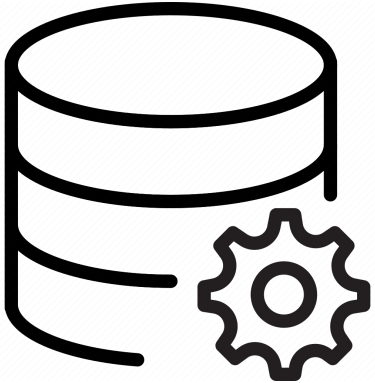
id	name	role	login_time
1	Alice	admin	2025-06-13 10:23
2	Bob	user	2025-06-13 11:02
3	Mallory	attacker	2025-06-13 11:06

```
SELECT * FROM users;
SELECT name FROM users WHERE role = 'attacker';
```



Вставка данных

```sql
INSERT INTO users (name, role, login_time)
VALUES ('Eve', 'attacker', '2025-06-13 12:00');
```

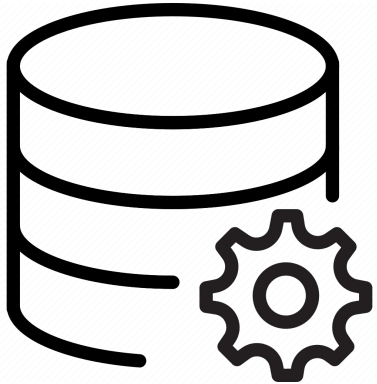


Вставка данных

Важно: нельзя вставлять данные **как есть из формы пользователя!**

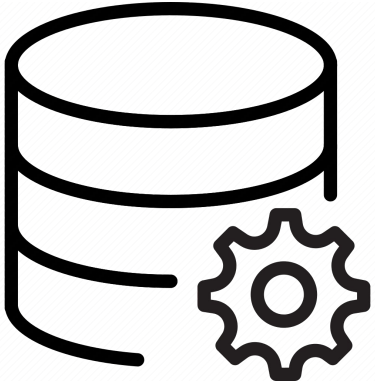
```
import sqlite3
conn = sqlite3.connect("db.sqlite")
cursor = conn.cursor()

name = input("Имя:")
cursor.execute("INSERT INTO users (name) VALUES (?)", (name,))
conn.commit()
```



Создание таблицы в Python

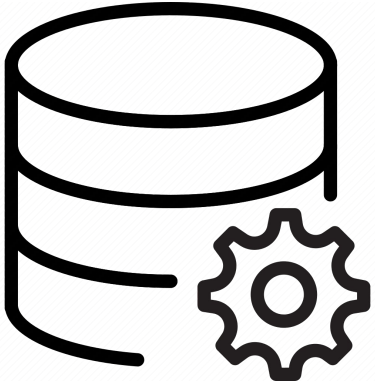
```
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    role TEXT,
    login_time TEXT
)
""")
```



Пример: логин-трекинг

- Добавляем нового пользователя
- Логируем время входа
- Ищем попытки атак

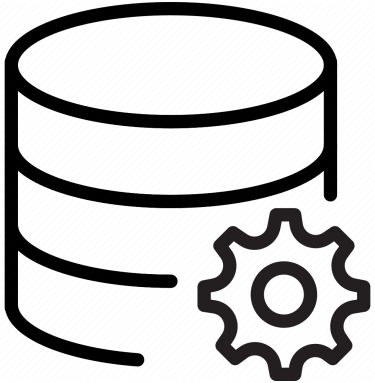
```
SELECT * FROM users WHERE role = 'attacker';
```



SQL-запросы в Python (всё вместе)

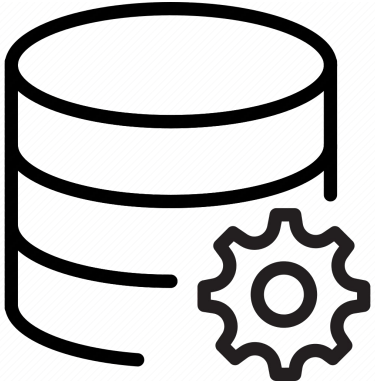
```
conn = sqlite3.connect("db.sqlite")
cursor = conn.cursor()

cursor.execute("SELECT name FROM users WHERE role='admin'")
admins = cursor.fetchall()
print(admins)
```



Типичные ошибки и риски

❌ SQL-инъекции (если не экранируешь ввод)
❌ Запрос без WHERE — удалит всё!
❌ Плохая структура таблиц (нельзя искать)
❌ Работа с устаревшими/публичными БД



Итоги урока

- Узнали как хранятся большие объёмы данных, а также что такое базы данных
- Узнали где специалист ИБ может с ними столкнуться
- Изучили какие проблемы в безопасности баз данных могут встретиться и как с ними бороться